

CyberGraph: Interactive Security Review with Code Knowledge Graphs and Evidence-Grounded Answers

Azizur Rahman¹, Laraib Hasan¹, Rushikesh Hiray¹, Shubham Srivastava¹,
Mohammad Hijjawi¹, Jafar Rahman², Harshit Taneja¹

¹University of Birmingham ²Independent Researcher
er.azizurrahman@gmail.com, laraib.hasan45@gmail.com, rhiray03@gmail.com
shubham03workplace@gmail.com, mohammad.hijjawi1997@gmail.com
rahmanjafar2004@gmail.com, hxt532@student.bham.ac.uk

Abstract

CyberGraph connects exposed routes, controls, and sensitive operations through a local security knowledge graph, an offline explorer, and evidence-grounded question answering. Language-specific analyzers populate a shared graph; bounded call traversal identifies potential entrypoint-to-sink paths. Retrieved records expose citations and heuristic confidence. Answers are deterministic by default, with optional language-model phrasing. We demonstrate a review loop on the public PyGoat training application: build the graph, inspect a SQL-injection path, ask a cited question, parameterize the query, and compare scans. The repair removes one finding while the system retains a REVIEW verdict. The demonstration emphasizes auditable evidence and explicit uncertainty rather than automatic claims of program safety.

Code — <https://github.com/AQ-Labs/cybergraph>

Motivation and Related Work

Reviewers need to connect a suspicious call to its entrypoint, controls, and source, then check what changes after a repair. CyberGraph integrates this workflow with graph-backed explanations, not a claim of stronger detection.

Code property graphs support vulnerability-oriented traversal (Yamaguchi et al. 2014); CodeQL provides data-flow analysis (GitHub 2026), and FlowDroid models Android lifecycle-aware taint propagation (Arzt et al. 2014). CyberGraph’s lightweight graph does not claim their analysis precision. Devign learns graph-based vulnerability classifiers (Zhou et al. 2019), while LineVul predicts vulnerable lines using transformers (Fu and Tantithamthavorn 2022). CyberGraph instead exposes extracted evidence for interactive review.

Retrieval-augmented generation conditions answers on retrieved information (Lewis et al. 2020). GraphRAG uses LLM-derived entity graphs and community summaries for corpus-level questions (Edge et al. 2025); CyberGraph retrieves typed code records and paths, without requiring LLM-based graph construction. Unlike work on LLM-generated vulnerability repairs (Pearce et al. 2023), this demo applies a predefined patch and inspects its effects. These are methodological distinctions, not head-to-head performance comparisons.

The contribution is an inspect–question–repair–recheck interaction: a developer explores a path, queries the same graph-derived evidence, and checks the effect of a patch. The AI-facing component is structured retrieval for optional neural explanation; neither extraction nor the demonstrated answers require a model. We assess what path records add through a small retrieval ablation, not a claim of a new detection algorithm or measured developer gains.

System Architecture

Extraction and storage. The Python CLI dispatches analyzers for Python, JavaScript/TypeScript, Go, Java, and C#. Python uses its abstract syntax tree; other adapters use lightweight structural and pattern-based extraction. Framework coverage varies. A shared contract stores nodes, edges, and findings in SQLite. Relationships include EXPOSES_ENTRYPOINT, GUARDS, SANITIZES, CALLS_RESOLVED, and REACHES_SINK. File/line metadata links evidence to source. Optional scanner imports add findings and advisories.

Path analysis and uncertainty. Call resolution links candidate definitions across files. Bounded breadth-first traversal follows these links from entrypoints to sinks. Paths retain the weakest call-resolution confidence, sanitizer markers, and available local taint evidence. These signals prioritize inspection, not prove exploitation; a function-level control need not protect every execution. Change checks distinguish failures, unknown results, and unsupported capabilities. Their ACCEPT/REVIEW decision is separate from the CI gate: ACCEPT concerns checks run, not whole-program safety.

Grounded answers and presentation. The `explain` command classifies questions by keywords and ranks records by category agreement and lexical overlap, favoring findings and paths. Records carry file/line or path citations. Local answers mark empty retrieval as insufficient evidence and related-only evidence as inconclusive. Confidence is heuristic, not calibrated. Optional Anthropic, OpenAI, or Kimi clients receive numbered evidence and citation instructions, not a guarantee against unsupported generation. Default answers are assembled locally. The offline HTML explorer uses Cytoscape.js (Franz et al. 2016) for security-layer views, search and severity filters, focused paths, and source drill-down. JSON, SARIF, and optional MCP integration expose the evidence to other tools. Suppressed findings can retain

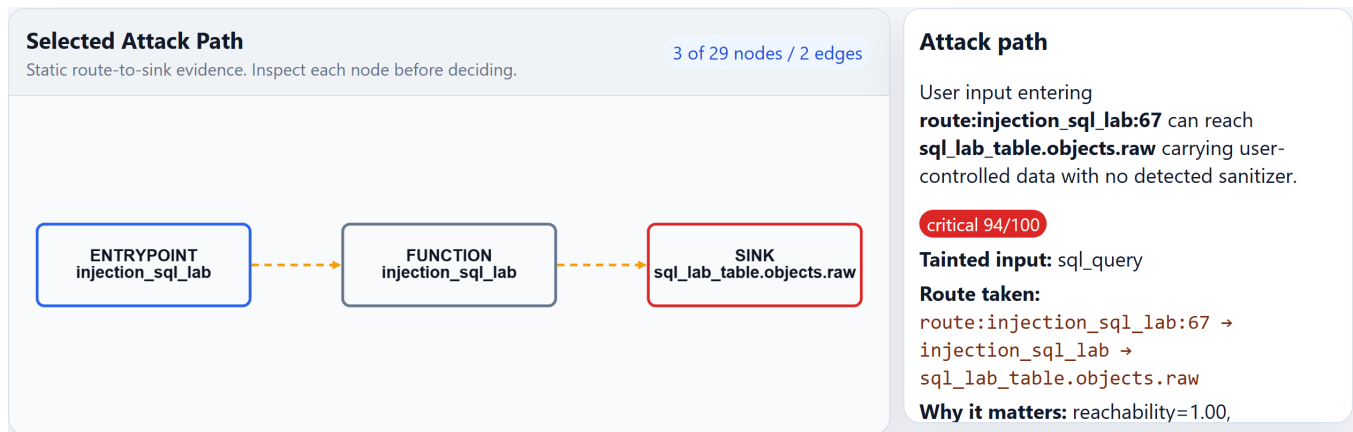


Figure 1: Actual offline explorer on PyGoat, with typography scaled for print. Selecting a path isolates the entrypoint, handler, and SQL sink; the scrollable evidence panel exposes its rationale. The displayed 94/100 is a review-priority heuristic, not an exploitation probability. Source drill-down supports verification.

underlying graph evidence for inspection.

Interactive Demonstration

The under-five-minute walkthrough uses PyGoat, an intentionally vulnerable Django training application (PyGoat Contributors 2026), at commit 19d17cc. The vulnerable application is neither started nor exploited. Figure 1 shows the running interface.

1. Build the graph. After installation and cloning the target, the presenter runs `cybergraph doctor .` and `cybergraph quickstart ..`. The latter creates configuration and a GitHub Actions workflow, with the database and `report.html` in `.cybergraph/`. Explicit exports produce `graph.json` and `findings.sarif`. Source snippets are opt-in because reports may disclose repository content.

2. Inspect a path. The baseline contains 806 stored nodes, 2,634 edges, and 14 findings. The presenter selects the SQL path from a Django route in `urls.py:67`, through `injection_sql_lab`, to `sql_lab_table.objects.raw` in `views.py:878`. The focused view isolates role-labeled nodes and directional edges. Source evidence shows request parameters at lines 858–859 and string-built SQL at line 864. Authentication at line 856 does not make this SQL construction safe. Attendees can inspect other paths and source.

3. Ask and repair. The presenter asks `explain` to trace the handler to the sink. The local response cites the path and sink line and recommends parameterization. The presenter replaces string concatenation with placeholders and a parameter list, then runs `cybergraph analyze .` and `cybergraph history ..`. Findings decrease from 14 to 13, with no new or regressed findings. The verdict remains **REVIEW: unsafe calls and configuration warnings remain**. A static path can persist after parameterization because structural reachability and a vulnerability verdict are different outputs.

Delivery. Attendees choose paths and questions on a lap-

top, inspect citations, and observe scan changes. Pinned code, prebuilt reports, and video provide an offline fallback. Model calls and CI uploads are optional and not demonstrated.

Evidence, Scope, and Limitations

Regression check. At revision b3b47b1 (analysis base 18329a6), the existing 11 seeded reachability cases yield 9 true positives, 0 false positives, and 0 false negatives by expected endpoint/sink label matching. Nine positives span Python, Go, and Java; two negatives cover Python and Go. This checks consistency, not every intermediate edge or real-world detection accuracy. Dataset artifacts can distort vulnerability evaluation (Chakraborty et al. 2022).

Retrieval ablation. For each of the nine positive cases, we ask “What sensitive sink is reachable from [entrypoint]?” using its existing expected label. Ranking is unchanged across three record pools, with at most two complete records and 2,000 characters per question. Both expected endpoint labels occur in retrieved evidence for 0/9 questions with findings only, 8/9 without path records, and 9/9 with path records. Only the last condition returns connected path citations (9/9). Mean context lengths are 136, 243, and 544 characters, respectively. These are seeded, template-generated questions: label co-occurrence is not relationship correctness, and the larger context is a confound. The saved protocol, per-question records, and runner make this descriptive evidence-availability comparison reproducible; it is neither a held-out benchmark nor an LLM evaluation.

The PyGoat loop is not a user study or new vulnerability discovery. The repair was syntax-checked, not database-tested. Dynamic dispatch, missing framework models, function-level control markers, and bounded traversal can produce missed or misleading paths. External LLMs receive selected context only when enabled; citation instructions do not guarantee faithful generation. Broader labeled-repository evaluation, model faithfulness tests, and usability studies remain future work.

References

- Arzt, S.; Rasthofer, S.; Fritz, C.; Bodden, E.; Bartel, A.; Klein, J.; Le Traon, Y.; Outeau, D.; and McDaniel, P. 2014. FlowDroid: Precise Context, Flow, Field, Object-Sensitive and Lifecycle-Aware Taint Analysis for Android Apps. In *Proceedings of the 35th ACM SIGPLAN Conference on Programming Language Design and Implementation*, 259–269.
- Chakraborty, S.; Krishna, R.; Ding, Y.; and Ray, B. 2022. Deep Learning Based Vulnerability Detection: Are We There Yet? *IEEE Transactions on Software Engineering*, 48(9): 3280–3296.
- Edge, D.; Trinh, H.; Cheng, N.; Bradley, J.; Chao, A.; Mody, A.; Truitt, S.; Metropolitansky, D.; Ness, R. O.; and Larson, J. 2025. From Local to Global: A Graph RAG Approach to Query-Focused Summarization. arXiv:2404.16130v2.
- Franz, M.; Lopes, C. T.; Huck, G.; Dong, Y.; Sumer, O.; and Bader, G. D. 2016. Cytoscape.js: A Graph Theory Library for Visualisation and Analysis. *Bioinformatics*, 32(2): 309–311.
- Fu, M.; and Tantithamthavorn, C. 2022. LineVul: A Transformer-Based Line-Level Vulnerability Prediction. In *Proceedings of the 19th International Conference on Mining Software Repositories*, 608–620.
- GitHub. 2026. About Data Flow Analysis. <https://codeql.github.com/docs/writing-codeql-queries/about-data-flow-analysis/>. Accessed September 17, 2026.
- Lewis, P.; Perez, E.; Piktus, A.; Petroni, F.; Karpukhin, V.; Goyal, N.; Küttler, H.; Lewis, M.; Yih, W.-t.; Rocktäschel, T.; Riedel, S.; and Kiela, D. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, volume 33, 9459–9474.
- Pearce, H.; Tan, B.; Ahmad, B.; Karri, R.; and Dolan-Gavitt, B. 2023. Examining Zero-Shot Vulnerability Repair with Large Language Models. In *2023 IEEE Symposium on Security and Privacy*, 2339–2356.
- PyGoat Contributors. 2026. PyGoat: Intentionally Vulnerable Web Application Security in Django. <https://github.com/adeyosemanputra/pygoat>. Commit 19d17cc8874861142b330636d068bbde54e86b85; accessed September 17, 2026.
- Yamaguchi, F.; Golde, N.; Arp, D.; and Rieck, K. 2014. Modeling and Discovering Vulnerabilities with Code Property Graphs. In *2014 IEEE Symposium on Security and Privacy*, 590–604.
- Zhou, Y.; Liu, S.; Siow, J.; Du, X.; and Liu, Y. 2019. Devign: Effective Vulnerability Identification by Learning Comprehensive Program Semantics via Graph Neural Networks. In *Advances in Neural Information Processing Systems*, volume 32.